

2025 年度 科学技術計算

学生番号 36714143
情報工学科知能情報分野
和田小百合

2026 年 1 月 20 日

目次

1	課題 07-2	2
1.1	課題の内容	2
1.2	chord 法について	2
1.3	実装方法について	2
1.4	実行結果	3
1.5	ニュートン法との比較	4
2	課題 07-4	6
2.1	課題の内容	6
2.2	LM 法について	6
2.3	SVD を用いた LM 法について	7
2.4	作成した関数について	7
2.5	実行結果について	9

1 課題 07-2

1.1 課題の内容

多次元 ($n = m$) のニュートン法の更新式の一つである chord 法の実装を行う課題です。

1.2 chord 法について

chord 法は初期値 $\mathbf{x}^{(0)}$ のヤコビ行列 $J(\mathbf{x}^{(0)})$ を利用し続ける更新式を持ちます。以下が chord 法のアルゴリズムです。

初期値 $\mathbf{x}^{(0)}$, $k = 0$ とする。収束するまで以下を反復します。

$$\mathbf{d}^{(k)} = -J(\mathbf{x}^{(0)})^{-1} \mathbf{f}(\mathbf{x}^{(k)})$$

すなわち、

$$J(\mathbf{x}^{(0)})\mathbf{d}^{(k)} = \mathbf{f}(\mathbf{x}^{(k)})$$

を解き、更新式は、

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}^{(k)}$$

です。

1.3 実装方法について

ニュートン法と唯一違うところが更新式について初期値のヤコビ行列を用いるところなので、授業資料にありましたニュートン法の実行コードの一部を変更して実装しました。以下が chord 法を実装する関数です。

```
1  # 法chord
2  def chord(
3      f: callable,
4      J: callable,
5      x0: np.ndarray,
6      maxiter: int = 100,
7      callback: callable = None,
8  ) -> np.ndarray:
9
10     """法chord
11
12     Args:
13         f (callable): 目的関数 f(x). ベクトル値関数.
14         J (callable): ヤコビ行列 J(x).
15         x0 (np.ndarray): 初期値.
16         maxiter (int, optional): 最大反復回数. デフォルトは, 100
17         callback (callable, optional): 各反復で呼び出されるコールバック関数. デフォルトは,
18             None
19
20     Returns:
21         np.ndarray: 解 \hat{x}
22
23     """
24     n = len(x0)
```

```

23     assert len(f(x0)) == n
24     assert J(x0).shape == (n, n)
25     J = J(x0)
26
27     x = x0.copy()
28     for k in range(maxiter):
29         f_x = f(x) # 現在の関数ベクトルの計算
30
31         # 解の更新方向を計算d
32         d = solve(J(x0), -f_x)
33
34         x = x + d # 解を更新
35
36         if callback:
37             callback(x)
38
39         if np.isclose(norm(f_x), 0):
40             return x
41
42     return x

```

1.4 実行結果

目的関数は授業資料に合ったものである `Function2D()` を用い結果を出力しました。また、プロットや、グラフに描写する関数についても授業資料に掲載されていた関数を利用しました。

授業資料の中で利用した関数

`Function2D()`(目的関数)

`CacheXk` と `PlotCacheXk()`→(解の各反復ごとに $x_k, |f(x_k)|$ を記録し、グラフに出力する関数)

以下が実行コードです。

```

1     f = Function2D()
2
3     callback_list = []
4
5     x0_list = [
6         np.array([-1.0, 1.0]),
7         np.array([1.0, 0.0]),
8         np.array([0.0, 0.0]),
9         np.array([-1.0, 0.0]),
10    ]
11    for x0 in x0_list:
12
13        cache_callback = CacheXk(f, x0, f"Newton's method from {x0}")
14
15        x = newton_method(
16            f,          # 目的関数
17            f.jac,      # ヤコビ行列
18            x0,         # 初期値
19            callback=cache_callback # コールバック
20        )
21        print("x", x)
22        print("fx", f(x))

```

```
23
24     callback_list.append(cache_callback)
25
26 plot_cache = PlotCacheXk(callback_list)
27 figs = plot_cache.plot_2d()
28 # figs = plot_cache.plot_2d(plot_mse=True)
```

1.5 ニュートン法との比較

まず、それぞれの初期値から最適解に収束しているかニュートン法と比較します。

- ニュートン法の実行結果

```
x [0.35324662 0.60608174]
fx [0. 0.]
x [0.35324662 0.60608174]
fx [-1.11022302e-16 -5.55111512e-17]
x [0.35324662 0.60608174]
fx [1.11022302e-16 0.00000000e+00]
x [0.35324662 0.60608174]
fx [0. 0.]
```

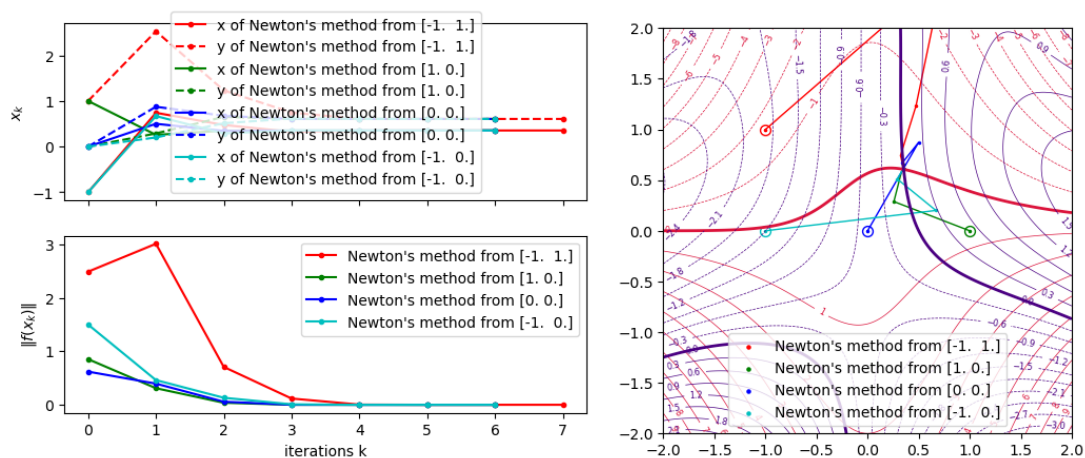
- chord 法の実行結果

```
x [0.35324662 0.60608173]
fx [ 4.02663836e-09 -2.11854773e-09]
x [0.35324662 0.60608173]
fx [ 2.14617379e-09 -1.62611202e-09]
x [0.35324662 0.60608174]
fx [-5.67704606e-09  3.81170728e-09]
x [0.35324662 0.60608174]
fx [ 1.39460310e-09 -5.69297243e-09]
```

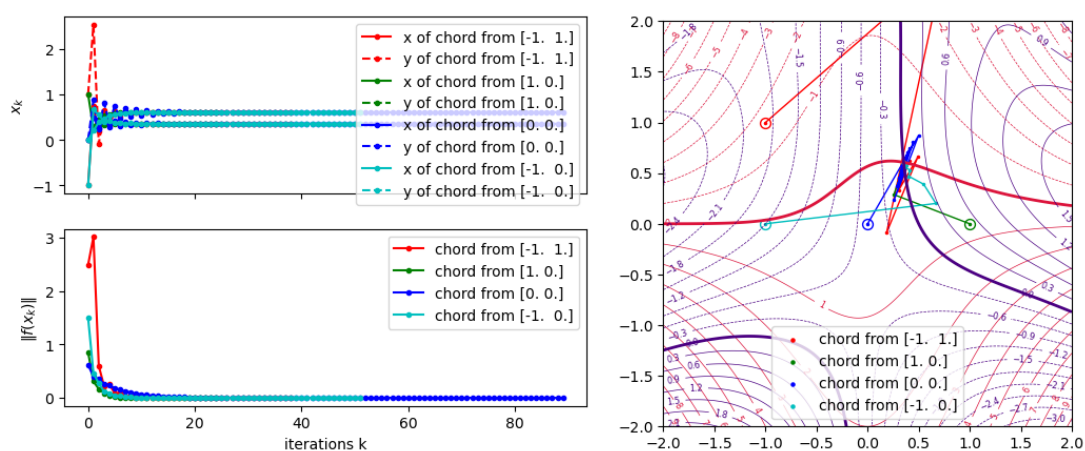
この結果から、同じ目的関数で収束した x_k の値が授業資料に掲載されているニュートン法と自分が制作した chord 法と一致しているため、chord 法はあっているといえるでしょう。

また、 $x_k, ||f(x_k)||$ の収束グラフも比較します。

- ニュートン法の $x_k, \|f(x_k)\|$ の収束グラフ



- chord 法の $x_k, \|f(x_k)\|$ の収束グラフ



この結果から chord 法は、ニュートン法よりも反復回数が多く、解を出力するのに時間がかかることがわかりました。また、更新式が初期値のヤコビ行列を使うため、近似解に近い値が出たとしても、その周辺で値が振動するので何回も回数を重ねないと買いにたどり着かないことがわかります。実際にニュートン法との反復回数の比較は以下の表となります。

表 1 収束結果

初期値	ニュートン法	chord 法
$(-1.0, 1.0)$	7	38
$(1.0, 0.0)$	6	24
$(0.0, 0.0)$	6	80
$(-1.0, 0.0)$	6	50

1.5.1 初期値を変更した場合

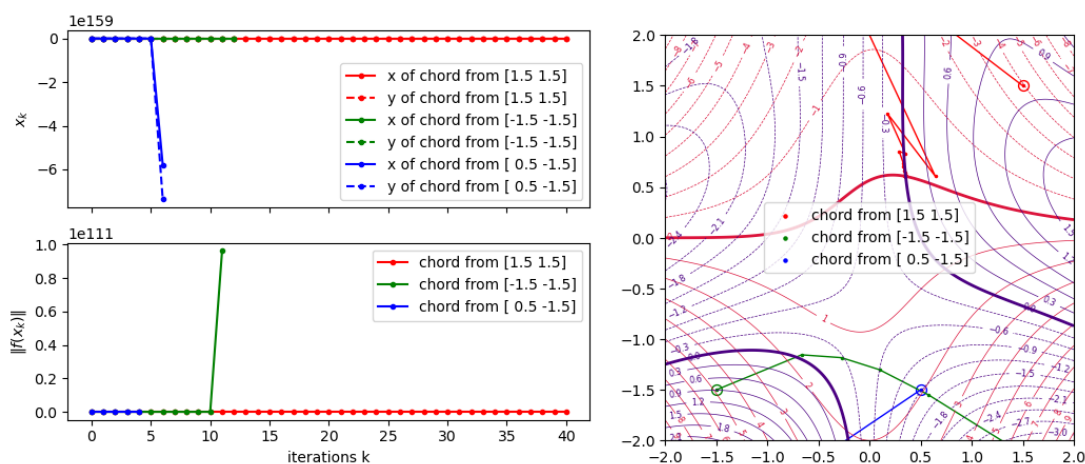
初期値を $(1.5, 1.5)$, $(-1.5, -1.5)$, $(0.5, -1.5)$ に設定して検証を行った。以下が実行結果である。

x [0.35324662 0.60608174]

fx [-3.89465693e-09 -1.04229864e-09]

```
x [nan nan]
fx [nan nan]
x [nan nan]
fx [nan nan]
```

また、 $x_k, \|f(x_k)\|$ の収束グラフは以下のようになります。



この結果から値からかけ離れている場合、別の等高線に反応してしまい、正しい値が得られず発散してしまうことがわかりました。

2 課題 07-4

2.1 課題の内容

SVD を用いて (疑似) 逆行列を計算する LM 法を実装する課題です。

2.2 LM 法について

LM 法のアルゴリズムは、以下のようになる。

収束するまで以下を反復

$$d^{(k)} = -(J^T J + \lambda^{(k)} I)^{-1} J^T f$$

すなわち、

$$(J^T J + \lambda^{(k)} I) d^{(k)} = -J^T f$$

を解く。更新式は、

$$x^{(k+1)} = x^{(k)} + \alpha d^{(k)}$$

ここで $\lambda^{(k)}$ は反復ごとに調整される正則化パラメータである。目的関数値が下がる場合には小さくし、そうでない場合には大きくするように更新します。 $\lambda^{(k)}$ を動的に変更する LM 法の実際のアルゴリズムは以下のようになる。

初期値

$$x^{(0)}, \lambda^{(0)}, k = 0$$

収束するまで以下を反復します。

$$d^{(k)} = -(J^T J + \lambda^{(k)} I)^{-1} J^T f$$

目的関数値が減少する ($\|f(x^{(k)})\| > \|f(x^{(k)} + \alpha^{(k)}d^{(k)})\|$) なら解を更新する。

$$x^{(k+1)} = x^{(k)} + \alpha^{(k)}d^{(k)}$$

$\lambda^{(k)}$ を減少させ、更新方向をニュートン方向に近づける：

$$\lambda^{(k+1)} \leftarrow \gamma_1 \lambda^{(k)}$$

目的関数値が減少しない ($\|f(x^{(k)})\| < \|f(x^{(k)} + \alpha^{(k)}d^{(k)})\|$) なら
解を更新しない：

$$x^{(k+1)} = x^{(k)}$$

$\lambda^{(k)}$ を増加させ、更新方向を最急降下方向に近づける：

$$\lambda^{(k+1)} \leftarrow \gamma_2 \lambda^{(k)}$$

$$k \leftarrow k + 1$$

ここで減少率 $\gamma_1 < 1$ と増加率 $\gamma_2 > 1$ の値は経験的に決定する（例えば $\lambda^{(0)} = 1, \gamma_1 = 0.8, \gamma_2 = 2$ ）

2.3 SVD を用いた LM 法について

ヤコビ行列の SVD を $J = U W V^T$ とすると、LM 法の右辺の行列部分は以下のように展開できる。

$$\begin{aligned} (J^T J + \lambda I)^{-1} J^T &= ((U W V^T)^T (U W V^T) + \lambda I)^{-1} (U W V^T)^T \\ &= (V W^2 V^T + \lambda I)^{-1} V W U^T \\ &= (V \text{diag}(\sigma_1^2 + \lambda, \dots, \sigma_p^2 + \lambda) V^T)^{-1} V W U^T \\ &= V \text{diag}(1/(\sigma_1^2 + \lambda), \dots, 1/(\sigma_p^2 + \lambda)) V^T V W U^T \\ &= V \text{diag}(1/(\sigma_1 + \lambda/\sigma_1), \dots, 1/(\sigma_p + \lambda/\sigma_p)) U^T \\ &= V W_\lambda^+ U^T \end{aligned}$$

ここで W_λ^+ は最後から 2 行目の、対角行列の部分を表す。

2.4 作成した関数について

実際に作成した関数は以下のようになります

```

1  # 法改良 LM
2  def levenberg_marquardt_svd(
3      f: callable,
4      J: callable,
5      x0: np.ndarray,
6      lmd: float = 1.0,
7      alpha: float = 0.5,
8      gamma1: float = 0.8,
9      gamma2: float = 2.0,
10     maxiter: int = 100,
11     callback: callable = None
12 ) -> np.ndarray:
13     """Levenberg-Marquardt 法
14
```

```

15     Args:
16         f (callable): 非線形方程式のベクトル値関数  $f(x)$ 
17         J (callable):  $f(x)$  のヤコビ行列  $J(x)$ 
18         x0 (np.ndarray): 初期値
19         lmd (float, optional): 正則化パラメータ  $\lambda$  の初期値. デフォルトは 1.0
20         alpha (float, optional): ステップサイズ. デフォルトは 0.5
21         gamma1 (float, optional):  $\lambda$  の減少率. デフォルトは 0.8
22         gamma2 (float, optional):  $\lambda$  の増加率. デフォルトは 2.0
23         maxiter (int, optional): 最大反復回数. デフォルトは 100
24         callback (callable, optional): 各反復ごとに呼び出されるコールバック関数. デフォルトは
        None
25
26     Returns:
27         np.ndarray: 解  $\hat{x}$ 
28     """
29     n = len(x0)
30     m = len(f(x0))
31     assert J(x0).shape == (m, n)
32
33     x = x0.copy()
34
35     for k in range(maxiter):
36         J_x = J(x) # ヤコビ行列  $J(x)$ 
37         U, w, V_T = np.linalg.svd(J_x, full_matrices=False)
38
39         W_plus = w / (w**2 + lmd)
40         W_plus_mat = np.diag(W_plus)
41
42         f_x = f(x) # 関数値  $f(x)$ 
43
44         # 解の更新方向を計算更新式d
45         d = - V_T.T @ W_plus_mat @ U.T @ f_x
46         x_next = x + alpha * d # 解を更新
47
48         if norm(f_x) > norm(f(x_next)): # 目的関数値が減少する場合
49             x = x_next # 解を更新
50             lmd *= gamma1 #  $\lambda$  を減少
51         else: # 目的関数値が減少しない場合
52             lmd *= gamma2 #  $\lambda$  を増加
53
54         if callback:
55             callback(x)
56
57         if np.isclose(norm(f_x), 0):
58             return x
59
60     return x

```

基本的な LM 法については、授業資料の関数を用いました。ヤコビ行列を SVD に分解して更新式を一部変更して実装しました。SVD について scipy の関数を用いて出力します。この時、 UWV の出力の V は、転置したものが出力されるので更新式で計算するとき、再度転置して計算を行いました。

2.5 実行結果について

以下のコードで目的関数の SVD を用いた LM 法の結果を出力しました。

```

1  f = Function2D(m=3)
2
3  callback_list = []
4
5  x0_list = [
6      np.array([-1.0, 1.0]),
7      np.array([1.0, 0.0]),
8      np.array([0.0, 0.0]),
9      np.array([-1.0, 0.0]),
10 ]
11 for x0 in x0_list:
12
13     cache_callback = CacheXk(f, x0, f"LM method from {x0}")
14
15     x = levenberg_marquardt_svd(
16         f,          # 目的関数
17         f.jac,      # ヤコビ行列
18         x0,         # 初期値
19         # alpha=1.0, # ステップサイズ
20         # lmd=1.0,   # λの初期値
21         callback=cache_callback # コールバック
22     )
23     print("x", x)
24     print("fx", f(x))
25
26     callback_list.append(cache_callback)
27
28 plot_cache = PlotCacheXk(callback_list)
29 figs = plot_cache.plot_2d(plot_mse=True)

```

このコードを実行した結果が以下のようになります。

```

x [0.36212601 0.23827601]
fx [ 0.30824088 -0.06369297  0.20829457]
x [0.36212601 0.23827601]
fx [ 0.30824088 -0.06369297  0.20829457]
x [0.362126   0.23827601]
fx [ 0.30824088 -0.06369298  0.20829457]
x [0.36212601 0.23827601]
fx [ 0.30824088 -0.06369297  0.20829457]

```

これは LM 法の結果

```

x [0.36212601 0.23827601]
fx [ 0.30824088 -0.06369297  0.20829457]
x [0.36212601 0.23827601]
fx [ 0.30824088 -0.06369296  0.20829457]

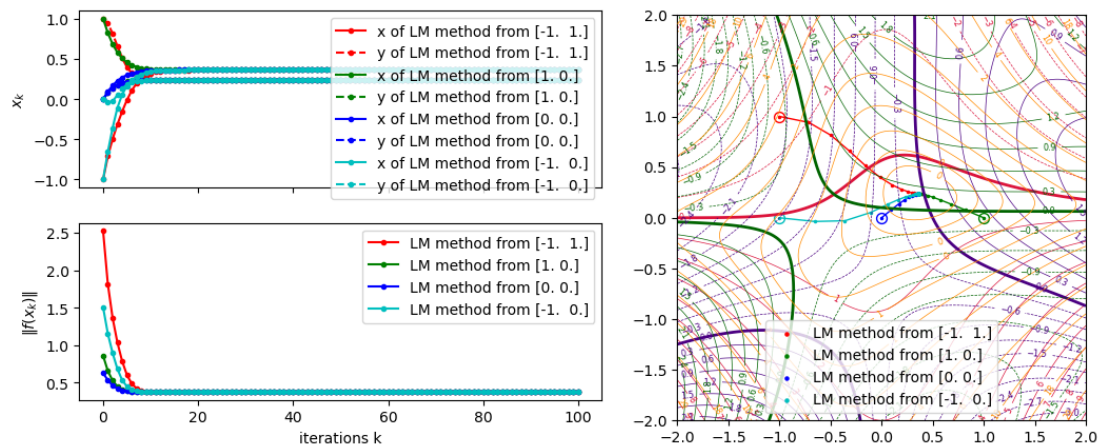
```

```

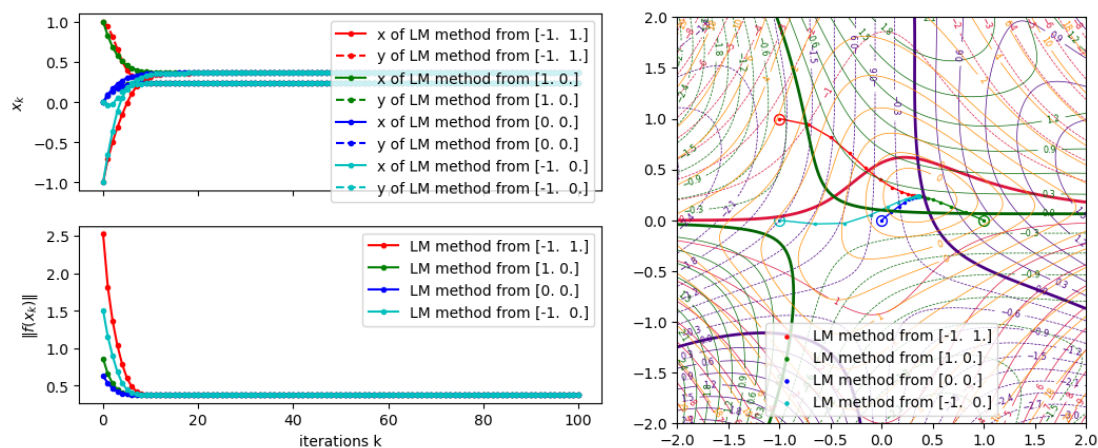
x [0.362126  0.23827601]
fx [ 0.30824088 -0.06369298  0.20829457]
x [0.362126  0.23827601]
fx [ 0.30824088 -0.06369297  0.20829457]

```

と一致するため、実装した関数があるとわかります。
また、収束をプロットしたものが以下になります。



また、LM 法の結果は授業資料より以下のような結果になっています。



このことから複数の初期値から最適解を導くことができているとわかりました。